Personal Memory

Project documentation

Requirements, system design, alternatives, and implementation roadmap

Language: English

Edition: 2026-10-01

Data boundary: GitHub and portfolio use synthetic data only. Personal conversations stay in the local system.

A human-readable guide to the purpose, requirements, architecture, existing alternatives, privacy boundaries, and phased delivery of the Personal Memory project.

Contents

Personal Memory: Requirements, Scope, and Project Rationale 4

1. Summary 4
2. Problem and User Context 4
3. Goals 4
4. Non-goals and Boundaries 5
5. Users and Scenarios 5
6. Functional Requirements 5
7. Non-functional Requirements 7
8. Architecture and Principles 7
9. Existing Products and Rationale for Building 7
10. Portfolio Value and Presentation 8

GitHub and Local-Use Data Policy 8

11. Delivery Phases and Acceptance Criteria 8
12. Risks and Questions to Validate 9
13. Decision History 10

Existing-Solutions Evaluation for Personal Memory 11

Project Requirements 11

Comparison 11

Conclusion and Decision 12

Portfolio Value 12

Sources Reviewed 12

Personal Memory System Design 13

Goal 13

Proposed Architecture 13

Data Separation 13

GitHub and Personal-Data Boundary 13

ChatGPT Import Without Assumptions 14

Search and Knowledge Graph 14

Interface 14

Incremental Delivery 14

Out of Scope for the First Version 14

Foundation Acceptance Criteria 14

Personal Memory Implementation Plan 15

Working Decisions 15

Phases and Acceptance Criteria 15

Dependency Order 17

Out of Scope for the First Slice 17

Personal Memory Checklist 18

Preparation 18

Phase 0 - Evaluate Existing Products 18

Phase 1 - Foundation and Contracts 18

Phase 2 - API and Lexical Search 18

Phase 3 - ChatGPT Import (when the archive is available) 18

Phase 4 - UI and Android on LAN 18

Phase 5 - Semantic Search 19

Phase 6 - MCP and Adapters 19

Phase 7 - Synchronization and Recovery 19

Personal Memory: Requirements, Scope, and Project Rationale

Version: 1.0 **Date:** 2026-10-01 **Status:** The user has confirmed the direction to build our own system where existing products do not meet the requirements; delivery is incremental. **Related documents:** System design, Existing-solutions comparison, Implementation plan, Checklist.

1. Summary

Personal Memory is a local-first personal system that brings conversations and other sources into a repository controlled by the user. The repository must remain useful independently of the AI provider or interface: ChatGPT, local models such as Qwen, gpt-oss-20b, and Gemma 4, agent harnesses such as OpenCode and Hermes, or a purpose-built interface.

The system preserves original sources separately from normalized content, indexes, and generated knowledge. It supports full-text and semantic search, navigation through conversations and relationships, and tracing every result back to its source. It is accessible from a Windows PC and Android. The PC is planned as the host and primary source of truth; the phone may hold a local copy or cache for offline use and later synchronization over the home network.

This is also a portfolio project. It documents a real need, evaluated alternatives, tradeoffs, data modeling, robust import, provenance, search, and integration with multiple AI clients. The goal is not to claim that a custom product is superior in every situation, but to show which concrete requirements remain unmet by existing solutions and how this system addresses them.

2. Problem and User Context

Useful history is scattered across conversations and tools. The built-in memory of one assistant does not give the user an independent repository that is easy to audit, export, and reuse with another agent. Basic search is insufficient when the user remembers the idea but not the exact wording; summaries without sources can distort context; and connecting to a home PC must account for the LAN and phone usage.

The user works on Windows, keeps projects under **D:\AI**, uses Android, and is usually at home with the phone and PC on the same network. The project has a GitHub repository: victorarsenie/PersonalMemory. A ChatGPT export archive has been requested and may arrive later. Its format is not yet available for inspection.

3. Goals

1. A user-controlled memory repository, with a canonical copy on the PC.
2. Byte-for-byte preservation of imported source files, with hashes and metadata.
3. Regenerable indexes and derived data, each linked to the source that supports it.
4. Lexical and semantic search, with filters and citable results.
5. Exploration of conversations, documents, entities, and relationships in a graph view.
6. Use from Windows and Android; LAN access in the first phase and phone offline support to the extent established by prototyping.
7. An API and adapters that let different applications and agents use the same repository.
8. ChatGPT import without assumptions about the export schema; implement the parser only after receiving and inspecting the real archive.
9. Include images and files attached to sources in the local repository, preserve originals, retain source links, and make them accessible on PC/Android.
10. Document the project's differentiation from existing products and present implemented capabilities honestly in the portfolio.

4. Non-goals and Boundaries

- We are not rebuilding a language model or replacing ChatGPT, local models Qwen, gpt-oss-20b, and Gemma 4, or the OpenCode and Hermes agent harnesses.
- We do not assume MCP integration is available in every ChatGPT app; verify the specific transport, plan, and client when integration work begins.
- We do not invent the ChatGPT export structure before inspecting it.
- We do not modify original archives or automatically promote extracted content to verified facts.
- We do not expose the PC directly to the internet or open router ports by default.
- We do not enable bidirectional synchronization without identifiers, versions, a journal, and explicit conflict handling.
- We do not build a native Android app immediately if an installable web client meets measured requirements; choose after prototyping.
- Core local search must not depend on a cloud service.

5. Users and Scenarios

5.1 Repository owner

- Adds or imports a source and reviews the ingestion report.
- Searches by words, concepts, time range, provenance, or entity.
- Opens a citation in the context of its original message or document.
- Reviews, confirms, or corrects derived entities and relationships.
- Uses the system from a phone on the same network; when the PC is unavailable, consults the available local copy and syncs later.
- Backs up the repository and can rebuild INDEX/DERIVED from RAW.

5.2 AI agent or external client

- Searches the personal repository through a stable API or MCP.
- Receives relevant passages with metadata and provenance, not unsupported assertions.
- Initially reads data only; any write operation requires explicit authorization and logging rules.

5.3 Later ChatGPT export import

- The user provides the archive.
- The system inventories files and identifies the actual structure without deleting or rewriting the source.
- The importer reports recognized, skipped, or unknown files and maps confirmed data to the canonical model.
- Retrying an import does not duplicate content; errors are explainable and recoverable.

6. Functional Requirements

F-01 - Sources and Imports

Each ingestion has an identifier, source type, import time, adapter version, and outcome. Sources may be archives, documents, images, attachments, or later imports. An import preserves originals, describes what it processed, and can be retried without creating duplicates.

F-02 - Data Separation

- **RAW:** source archives and original image/file binaries, read-only after ingestion, preserved byte-for-byte and identified by SHA-256.
- **INDEX:** canonical model, file metadata, and links between attachments, messages, and sources; regenerable lexical/vector indexes.

- **DERIVED:** passages, extracted text, thumbnails, OCR, embeddings, summaries, entities, relationships, and claims. Each derived result retains provenance and processor/model version.

Reindexing may delete and rebuild only regenerable structures; it must not alter RAW.

F-03 - Provenance and Audit

Every result or claim must point to the source, conversation/document, message/passage, and position where the format allows. Distinguish original content, automatically extracted content, user-confirmed content, and disputed/corrected content. Preserve transformation and synchronization history.

F-04 - Search

Lexical search supports phrases and filters by date, source, conversation, entity, and type. Semantic search uses embeddings through a configurable provider, preferably local, and can be rebuilt when the model changes. Combined results show relevant passages, useful scores, metadata, and links to full context.

F-05 - Knowledge Graph

The graph stores entities, relationships, and claims as derived data with provenance. The interface lets users navigate from a concept to the conversations and passages that support it. Automatically extracted content remains marked as such until confirmed; contradictions are not silently overwritten.

F-06 - API and MCP

The versioned API is the shared contract for the UI and integrations. MCP initially exposes search and read operations; every result includes provenance. Agent writes are enabled separately after approval, validation, and change-log rules are defined. Adapters for local models Qwen, gpt-oss-20b, and Gemma 4, plus ChatGPT, OpenCode, and Hermes clients/agents, are optional components and do not change the data model.

ChatGPT limitation: OpenAI's official documentation checked on 2026-10-01 says MCP apps are available on web, not mobile; ChatGPT connects to remote MCP servers, not directly to a local server. A server on a private network needs a Secure MCP Tunnel for supported OpenAI products. Developer mode and full MCP actions depend on plan and workspace. Therefore, a purpose-built Android UI/API remains necessary; ChatGPT integration is an optional web route, not the only mobile interface. OpenAI: Developer mode and MCP apps in ChatGPT.

F-07 - PC, Android, and Synchronization

The PC hosts the service and primary copy. Android connects on the LAN and may hold a local replica/cache, including synced attachments for offline access. Synchronization transfers identified/versioned changes and files, can resume interrupted transfers, and detects conflicts without losing variants. Choose the exact protocol and PWA versus native client after prototyping and validating offline requirements.

F-08 - Interface

The interface provides search, result lists, conversation/document views with passage navigation, an entity/relationship map, provenance inspection, synchronization status, and an import flow. It previews supported images/files in source context and provides access to originals. It must make the distinction between source and inference visible.

F-10 - Images and File Attachments

The system imports and preserves image/file attachments from conversations and documents, including formats without an available extractor. Keep each original intact in RAW and record its hash, name, observed MIME type, size, and provenance to the source/originating message where available. The attachment remains accessible from its conversation context on PC and Android; the interface can display images or supported previews and open/download the original.

Extract and index content only for formats supported by configured processors. For images, OCR, visual description, and visual embeddings are optional derived capabilities, preferably processed locally. Unknown or unreadable files must not block import: preserve and link them to their source, and make them discoverable by name/metadata. Thumbnails, OCR, and extracted text are rebuildable and never replace originals. Transfer files to the Android replica with resumable transfers and offline access to synced items.

Acceptance: the original can be opened/recovered with an identical hash after import; the attachment has a navigable source link; supported content is searchable; unknown types are preserved and reported.

F-09 - Backup, Privacy, and Portability

Backups include RAW, the canonical database, and the configuration needed for restoration; regenerable indexes may be rebuilt. LAN access is authorized; public ports are not exposed by default. Export formats and schema are documented so the user is not locked into one client.

7. Non-functional Requirements

- **Local-first:** core storage and search run on the user's devices; cloud connectivity is not required for core functions.
- **Source safety:** operations on INDEX/DERIVED do not alter RAW files.
- **Traceability:** results and relationships are explainable through provenance.
- **Interoperability:** AI clients can change without migrating the canonical repository into a proprietary memory service.
- **Reproducibility:** migrations and initialization/import operations are versioned and documented.
- **Incremental evolution:** each phase produces a usable slice with stable contracts and clear migration.
- **Portability:** schema and exports are documented sufficiently for backup, audit, and migration.

8. Architecture and Principles

8.1 Components

1. **Local data store:** RAW files and canonical database on the PC.
2. **Ingestion/import adapters:** parse known sources and report unknowns.
3. **Search/index service:** full-text and later semantic indexes.
4. **Versioned local API:** used by the UI, phone, and clients.
5. **Exploration UI:** initially a browser/installable web app; validate PWA versus native separately.
6. **Android replica/sync:** incremental feature with offline cache and explicit conflict resolution.
7. **AI adapters:** MCP, embeddings, and client-specific integrations isolated from the core.

8.2 Initial Domain Model

Candidate entities: **Source**, **ImportRun**, **Conversation**, **Message**, **Attachment**, **Document**, **Passage**, **Entity**, **Claim**, **Relation**, **Provenance**, **Embedding**, **SyncEvent**, and **Revision**. Exact identifiers, schema, and states are set in an architecture decision before implementing the core.

8.3 Design Principles

- The source of truth resides in the user's repository, not an assistant's index.
- RAW is not a cache and cannot be recreated from normalized data.
- INDEX and DERIVED are rebuilt with explicit parser/model versions.
- Relationships and derived memories retain the citations that support them.
- Interfaces are adapters; data contracts are independent of them.
- Choose technologies based on the user's Windows environment and document dependencies, not just familiarity.

9. Existing Products and Rationale for Building

Products evaluated include Khoj, Open WebUI, AnythingLLM, Memos, Mem0, and Letta. They provide useful capabilities for personal knowledge management, RAG, agent memory, graph memory, or MCP; the project should not automatically reimplement their UI or runtime.

The comparison document links to official documentation and phrases gaps as “not identified in the reviewed documentation.” The working hypothesis is that none of the documented products combines, in one user-controlled

repository: RAW preservation of the real archive, ChatGPT import adapted after inspection, granular provenance, rebuildable indexes, an explorable graph, hybrid search, and interoperability across PC/Android/agents.

Therefore the approved direction is **to build the core that addresses these gaps and reuse existing products when doing so reduces effort without compromising data ownership and provenance**. If prototyping shows an alternative covers more than expected, reduce the custom scope. Revisit this decision based on evidence, not attachment to custom implementation.

10. Portfolio Value and Presentation

The project can demonstrate:

- problem analysis and evaluation of alternatives;
- local-first architecture with clear RAW/INDEX/DERIVED responsibilities;
- a resilient, idempotent, explainable ingestion pipeline;
- provenance and graph/data modeling;
- lexical and semantic search with local models such as Qwen, gpt-oss-20b, and Gemma 4;
- API/MCP and multi-agent integration;
- PC/Android access, LAN, offline sync, and conflict handling;
- privacy, backup, and portability decisions;
- incremental delivery and technical documentation.

Demos, screenshots, fixtures, and all data in the GitHub repository use synthetic data only. The user's real conversations and archive are used only locally in the Personal Memory instance. Never publish or commit archives, RAW, extracts, indexes, embeddings, logs, screenshots, or results containing personal data. Do not claim a feature is complete until there is a verifiable demonstration.

GitHub and Local-Use Data Policy

- The GitHub repository, including commits and portfolio materials, contains only code, documentation, and synthetic data created specifically for demonstrations.
- Synthetic data is not produced by anonymizing real conversations; generate it separately to avoid leaks through details or metadata.
- The ChatGPT archive and personal conversations are imported only into the user-configured local instance. Never copy them into Git-tracked directories.
- RAW, INDEX, DERIVED, backups, ingestion logs, and artifacts generated from personal sources stay local and are excluded from Git. Configure the repository to enforce this before importing real data.
- Prototypes, walkthroughs, and public materials use synthetic data. Importer correctness may also be checked locally against a copy of the real archive without publishing content, results, or screenshots.
- Errors and logs must not include message text or private file content by default.

11. Delivery Phases and Acceptance Criteria

Phase 0 - Evaluate and Prototype Existing Products

- Complete the documented evaluation and practically check the closest candidates using synthetic data.
- Decide whether an existing product becomes the UI/client, whether to extend a product, or whether the custom core remains justified.
- Acceptance: a criteria-based decision with observed gaps; involve the user when scope changes materially.

Phase 1 - Data Foundation

- Establish repository instructions, runtime, RAW/INDEX/DERIVED layout, Git-ignored configuration, database, and migrations.

- Acceptance: an empty repository can be initialized; schema is versioned; reindexing demonstrably leaves RAW unchanged.

Phase 2 - API and Lexical Search

- Versioned API, controlled source/import operations, FTS, and results with provenance.
- Acceptance: synthetic content can be added, searched, and opened in its source context.

Phase 3 - ChatGPT Importer

- After the archive arrives, inventory real formats, including how images and file attachments are represented; preserve the raw archive locally and implement adapters for observed formats.
- Acceptance: complete report, retryable import, unknowns preserved/reported, verifiable provenance, and attachments retained with links to their messages; validation against a copy of the real archive runs locally and is not included in Git.

Phase 4 - PC/Android UI and Graph

- Search, context, provenance, graph exploration, LAN access, and incremental cache/sync.
- Add image/file gallery and supported previews, access to originals, and offline availability for synced attachments.
- Acceptance: demonstrated use on PC and Android; files open from their source message, and conflicts/incomplete transfers are visible.

Phase 5 - Semantic Search and Local Models

- Integrate local models such as Qwen, gpt-oss-20b, and Gemma 4 through compatible runtimes.
- Configurable local embedding provider, model/version metadata, lexical/semantic fusion over text extracted from supported formats.
- Optional local OCR/visual description/embeddings can be added as derived processing without replacing original images.
- Acceptance: semantic results point to identifiable passages/attachments; re-embedding does not alter RAW.

Phase 6 - MCP and Clients

- Initially read-only MCP; test each target client and document transport restrictions.
- Acceptance: search/read returns sources and provenance; writes remain disabled until explicitly authorized.

Phase 7 - Backup, Security, and Hardening

- Backup/restore procedure, LAN authorization, logs, and optional remote access through a private network.
- Acceptance: restoration is demonstrated; no default public exposure.

12. Risks and Questions to Validate

- **Unknown export format:** defer the parser until the real archive; start with the adapter interface and contract.
- **Different MCP support per client:** test at integration time; ChatGPT MCP apps are web-only and do not connect directly to a local server according to reviewed OpenAI documentation. Keep a separate Android client; consider a private tunnel only for supported OpenAI surfaces.
- **Offline and replication complexity:** prototype and measure before choosing PWA/native and a sync protocol.
- **Inaccurate inferences:** use verification states and citations; the user retains control of facts.
- **Scope growth:** deliver phases with acceptance criteria and reuse existing products where central requirements are preserved.
- **Local model availability:** the core works with lexical search without embeddings; configure semantic search later for Qwen, gpt-oss-20b, Gemma 4, or other compatible models.

13. Decision History

- **2026-10-01:** The user confirmed Windows PC and Android as target devices, usually on the same home network, the repository under **D:\AI\PersonalMemory**, GitHub for versioning, a later ChatGPT archive, and future integration with local models Qwen, gpt-oss-20b, and Gemma 4, plus OpenCode and Hermes clients/agents.
- **2026-10-01:** The user confirmed building our own system if existing products do not meet the requirements, and requested documenting the comparison and added value as a portfolio project.
- **2026-10-01:** OpenAI documentation confirms MCP apps are web-only, connect to remote servers, and may require a Secure MCP Tunnel for private-network servers; direct mobile MCP access is not assumed.
- **2026-10-01:** The user specified that GitHub will contain synthetic data only for possible open-source publication; personal conversations will be used only locally in the created instance.
- The ChatGPT export has not been inspected; its schema remains unknown.

Existing-Solutions Evaluation for Personal Memory

Checked: 2026-10-01 **Purpose:** determine whether an existing product covers the need or whether a custom project brings technical and user value. **Note:** this comparison describes documented capabilities. A feature missing from the documentation reviewed is not proof that the product cannot provide it through extensions.

Project Requirements

- Windows PC hosts the local source of truth; Android can access it on the home network.
- Imported original data remains intact, separate from indexes and derived data.
- Lexical and semantic search results include verifiable provenance to the source conversation/document.
- Conversations and knowledge can be explored through search and a relationship map.
- The ChatGPT importer is written after inspecting the real archive, without assumptions about its format; image/file attachments must be preserved and linked to their source message.
- The core can later serve local models such as Qwen, gpt-oss-20b, and Gemma 4, as well as clients/agents such as OpenCode, Hermes, and ChatGPT through stable adapters.
- Imports, indexes, and derived memory can be rebuilt from preserved sources; mobile access and backup have clear behavior.

Comparison

Product	Documented strengths	Gaps against Personal Memory
Khoj	Self-hosted personal AI/second brain; chat and search over files; browser and app access; local or cloud models. Docs · Self-host	I did not identify documented support for a dedicated ChatGPT archive importer with immutable RAW and per-message provenance, or a provenance-aware conversation graph. It is the closest candidate for a personal self-hosted prototype.
Open WebUI	Knowledge bases/RAG, models through Ollama and compatible APIs, local options, and connections to MCP servers over Streamable HTTP. Knowledge · MCP	Primarily a chat interface and retrieval layer for document collections. The reviewed documentation does not describe a single managed workflow for an immutable ChatGPT archive, message-level provenance, and graph exploration. It may work as a UI/model client over a separate backend.
AnythingLLM	Flexible document management, attached or embedded RAG documents, and local or cloud model choices. Documents · Models	Workspace/RAG does not by itself provide a canonical conversation repository with preserved RAW, fine-grained provenance, and a relationship graph. It may be evaluated as an existing UI/RAG component rather than assumed to be a complete replacement.
Memos	Self-hosted note-taking application with built-in MCP server and search/create/update operations for memos. MCP	Useful for personal notes and MCP, but the documented model is memo-centric. I did not find the full conversation archive import, hybrid search, and provenance-aware graph requested here.

Product	Documented strengths	Gaps against Personal Memory
Mem0	Agent memory layer with add/search API, semantic memory, and graph-memory options. Overview · Graph memory	Solves agent memory (selected/retrieved facts), not full preservation and exploration of original conversations with RAW and per-result provenance. Useful later as a reference or adapter, not as the archive repository.
Letta	Stateful agent runtime with persistent memory, self-hosted App Server, and MCP integration in its ecosystem. App Server · MCP	Agent-centered memory and conversations. Our need is an interoperable personal repository that stays canonical regardless of which agent or client is in use.

Conclusion and Decision

None of the products reviewed documents, in a single installation, the complete requested workflow: ChatGPT archive import after inspecting its real format, intact RAW preservation, rebuildable indexes, granular provenance, lexical and semantic search, relationship exploration, and shared use from PC/Android and MCP adapters.

Building a custom system is therefore justified, with two scope constraints:

1. Do not rebuild generic components that can be reused. Evaluate existing products as UIs or model/MCP clients, especially Khoj and Open WebUI.
2. Build the distinctive core: canonical RAW/INDEX/DERIVED repository, importer based on the real archive, provenance, graph, and interoperable API/adapters.

Portfolio Value

The project demonstrates how to validate a need, compare existing products, define clear boundaries between raw data and indexes, implement repeatable import and provenance, integrate lexical/semantic search, provide cross-device access, and expose API/MCP to multiple agents. Documentation will include rationale, alternatives, architecture, decisions, and demonstrated limitations; it will not claim advantages that have not been implemented and verified.

Sources Reviewed

- Khoj: Overview, Self-host
- Open WebUI: Knowledge bases, MCP
- AnythingLLM: Documents, Model configuration
- Memos: MCP server
- Mem0: Memory overview, Graph memory
- Letta: Self-hosted App Server, MCP server

Personal Memory System Design

Status: Direction confirmed by the user on 2026-10-01 **Date:** 2026-10-01

Goal

Incrementally build a personal memory system accessible from Windows and Android. The PC hosts the services and primary copy; the phone keeps a local, searchable-offline copy that syncs with the PC when both are on the same Wi-Fi network. Remote access may be added later through a private network such as Tailscale.

The system preserves sources intact, including images and attached files, shows the provenance of each piece of information, and can later serve local models such as Qwen, gpt-oss-20b, and Gemma 4, as well as agent harnesses such as OpenCode and Hermes. ChatGPT integration is a separate, optional target. As of 2026-10-01, OpenAI documentation says MCP apps are web-only, not mobile; ChatGPT connects to remote MCP servers, and a server on a private network may need a Secure MCP Tunnel. Access and available actions depend on plan/client. The purpose-built Android app remains the direct mobile route. OpenAI source.

Full requirements, scenarios, acceptance criteria, and portfolio rationale are in the requirements document. The product comparison is in the existing-solutions evaluation.

Proposed Architecture

- **Windows host:** local service with primary database, versioned API, and search. It listens only on the configured local network; access requires pairing/authorization for the phone. No router port forwarding by default.
- **Android:** installable app with a local database/cache for offline search and a sync queue. Choose the first client-installable PWA or native app-after checking offline needs and the available environment.
- **Synchronization:** the protocol transfers identified changes with versions and provenance between phone and PC. Conflicting changes are not silently overwritten; both variants are preserved and marked for resolution.
- **AI clients:** the API is the shared contract for apps. A separate MCP adapter initially exposes limited search/read operations and later controlled writes. Enable ChatGPT integration only on surfaces/plans that support it.

Data Separation

- **RAW:** original archives and imported binaries, including images and attachments, preserved byte-for-byte and identified by hash. No normalization or editing modifies them.
- **INDEX:** normalized data for search and synchronization, with references to originating RAW files and records.
- **DERIVED:** extracted text, thumbnails, OCR, chunks, embeddings, entities, relationships, summaries, and inferred claims. These can be rebuilt or marked reviewed without modifying RAW; visual processing is optional and preferably local.

The source-of-truth schema will include sources and imports, conversations/documents, messages, attachments, people/entities, claims and relationships, provenance, search passages, and sync events. Extracted facts link to the exact source and passage supporting them, and retain their state (derived, verified, or disputed).

GitHub and Personal-Data Boundary

The GitHub repository and portfolio use synthetic data only. The user's real conversations and ChatGPT archive are used only in the local Personal Memory instance. Keep RAW, INDEX, DERIVED, backups, ingestion logs, and outputs derived from private data outside tracked Git content. Configure exclusions before importing the real archive. Generate synthetic demo data independently rather than anonymizing the user's conversations. Importer validation against a copy of the actual archive may happen locally; its content and outputs are not published.

ChatGPT Import Without Assumptions

Do not encode an anticipated export format. The importer has an adapter interface and a staged workflow: inspect the real archive (file list and observed structure), report its contents, map observed formats to a canonical model, then perform an idempotent import. Copy the original archive into RAW before processing. Unknown items are reported and preserved, not discarded. Import does not begin until the user provides the archive and its format has been inspected.

Search and Knowledge Graph

The first search phase uses full-text and filters by source, date, and entity. Semantic search uses locally generated embeddings through a configurable model adapter; the vector index is derived and rebuildable. The knowledge graph stores entities, relationships, and claims together with provenance. Automatic extraction is not treated as a verified fact without confirmation.

Interface

The Windows/Android interface provides search, sourced results, conversation/document and attached image/file views, entity and relationship exploration, sync status, and provenance inspection. Editing and claim verification are explicit actions. Android uses the same API and provenance conventions as AI services.

Incremental Delivery

1. **Data contract and local core:** schema, migrations, RAW/INDEX/DERIVED directories, provenance, and Windows configuration.
2. **Search and API:** full-text search, versioned endpoints, and shared client contract.
3. **Android client and LAN sync:** local cache, pairing, synchronization, and offline search, including transfer of attachments for offline access.
4. **Exploration and graph:** search/map interface and provenance-aware claims.
5. **AI adapters:** MCP for local clients, local embeddings, and ChatGPT integration where supported.
6. **ChatGPT import:** after the archive arrives, inspect the real conversation, image, and file-attachment formats and implement an appropriate adapter.

Each slice remains usable and additive. Imports, indexes, and derived data must be rebuildable without losing the raw source.

Out of Scope for the First Version

- Public access to the PC, open router ports, or sync through a third-party cloud.
- Assumptions about the ChatGPT archive structure.
- Automatically promoting summaries/entities to verified facts.
- Agent writes without provenance and explicit authorization rules.

Foundation Acceptance Criteria

- A source can be preserved in RAW and traced through its hash and provenance metadata.
- Indexes and DERIVED data can be rebuilt without modifying RAW.
- The canonical model and API do not depend on one export format or one AI model.
- The architecture clearly distinguishes the primary PC copy, Android replica, and sync conflicts.
- ChatGPT mobile is not presented as connected over MCP while that feature is unsupported.

Personal Memory Implementation Plan

Date: 2026-10-01 **Basis:** System design **Status:** The user approved evaluating existing solutions and building our own where needed on 2026-10-01; implementation proceeds incrementally.

The detailed requirements, scope, and portfolio rationale are in the requirements document.

Working Decisions

- The local PC repository is the source of truth; raw imports remain unchanged in RAW.
- INDEX contains normalized data and rebuildable indexes; DERIVED contains linked relations, summaries, and vectors with source provenance.
- Before writing code, compare existing products against requirements and try configuring the best fit. Build custom components only for confirmed gaps. If a custom core remains necessary, the initial option is SQLite/FTS5 and a Python/FastAPI API; choose the runtime after inspecting the available environment.
- The ChatGPT importer is modular: define the adapter contract first, then implement a concrete parser only after receiving and inspecting the real archive.
- For Android, begin with an installable web interface accessible on the LAN. Decide on bidirectional offline sync and native app after validating the prototype and actual usage requirements.
- Connect local models Qwen, gpt-oss-20b, and Gemma 4, plus ChatGPT, OpenCode, and Hermes clients/agents, through adapters over the stable API and data model; none is a core dependency.
- Every phase preserves provenance (source, locator, import time, transformation, and related identifiers) and allows reindexing without changing RAW.

Phases and Acceptance Criteria

0. Evaluate Existing Solutions (before implementation)

The initial comparison is documented in Existing-solutions comparison. The user confirmed that custom development is appropriate if existing products do not meet the requirements, and that the rationale/comparison should be included in portfolio documentation.

- Practically evaluate Khoj, Open WebUI, AnythingLLM, and Memos against priority requirements: local data, local models, Android/browser on LAN, archive import, RAW preservation, provenance, relationship map, and MCP.
- For one or two candidates, review current documentation and create a prototype with synthetic data; do not import the real ChatGPT archive before inspecting it.
- Acceptance: clear comparison and recommendation: (a) configure an existing product, (b) use it as UI over a custom repository/API, or (c) implement the core, explaining the gaps that justify the effort.
- Verification: manual walkthrough on PC and Android where possible; record unmet requirements and any ChatGPT/MCP access limitations.
- The user approved a custom core with reuse of existing products where helpful; refine the specific product choices and adapters after practical evaluation.

1. Foundation and Data Contracts

- Inspect the environment and existing documentation; record actual runtime choices and directory structure.
- Define **RAW/**, **INDEX/**, **DERIVED/**, local configuration, and backup rules.
- Define the initial schema for sources, conversations, messages, attachment binaries, entities, relationships, and provenance; use versioned migrations and store hashes, filenames, MIME types, and sizes.
- Acceptance: an empty repository can be initialized reproducibly; transformations and reindexing never overwrite source data.
- Verification: manually review the schema, directories, and initialization instructions.

2. Local API and Lexical Search

- Add versioned local endpoints for service health, sources, search, and reading provenance-linked items.
- Configure SQLite FTS5 for lexical search, filtering, and navigation back to source.
- Add a documented generic import for text content and metadata without claiming compatibility with an uninspected ChatGPT export.
- Acceptance: the user can add, search, and open indexed content; each result identifies its source and location.
- Verification: manual walkthrough using clearly labeled synthetic data, not real personal data. Committed/published prototypes and fixtures use synthetic data only.

3. Modular ChatGPT Importer

- Define the adapter interface, import report, and policies for duplicates, attachments, and errors.
- When the archive is available, inspect its structure and included documentation before implementing the concrete parser.
- Inventory the attachments actually present; preserve originals byte-for-byte, link them to source messages, and report unknown formats without stopping import.
- Preserve original files exactly; write extracted content and indexes only to their designated areas.
- Acceptance: import is retryable/idempotent, reports accepted/rejected content, and allows indexes to be rebuilt from RAW.
- Verification: privately validate locally against a copy of the real archive, checking item counts and provenance; data and results never enter Git.

4. Exploration Interface and Android LAN Access

- Build UI for search, conversation/source navigation, and provenance inspection.
- Add graph visualization as an exploration layer over approved/derived entities and relationships.
- Configure controlled local access, clear PC/phone instructions, and useful behavior when the server is unavailable.
- Add previews for supported images/files, access to originals, and resumable transfers for attachments synced for offline use.
- Acceptance: PC and phone on the same network can access the system; UI shows sources and links without exposing raw directories.
- Verification: manual walkthrough on PC and Android over the local network.

5. Semantic Search and Local Integration

- Integrate local models such as Qwen, gpt-oss-20b, and Gemma 4 through compatible runtimes; configure embeddings through a separate replaceable provider with version metadata.
- Store vectors only in INDEX/DERIVED and link each vector to the model version and source text.
- Add semantic search and fusion with FTS without requiring a cloud model.
- Index text extracted from supported formats; treat OCR, visual description, and visual embeddings as optional local processing in DERIVED.
- Acceptance: changing the model allows re-embedding without touching RAW; results preserve provenance.
- Verification: manual walkthrough with an available local model; if unavailable, clearly document the deferred functionality.

6. MCP Bridge and Agent Adapters

- Publish an MCP bridge with a restricted initial read-only set: search, read, map, and provenance.
- Keep adapters for ChatGPT and the OpenCode and Hermes agent harnesses separate from the central server and schema; they can call local models such as Qwen, gpt-oss-20b, or Gemma 4.
- Add write operations only after authorization, validation, and change logging are defined.

- Acceptance: supported clients can find and read the same data through API/MCP, with each result's source visible.
- Verification: manually connect each client after checking its support and configuration.

7. Synchronization, Security, and Backup

- Evaluate a PC-authoritative model with local Android replica/cache and queues for offline changes.
- Document LAN authentication, minimal port exposure, backup, and restoration.
- Consider remote access only on request through a private tunnel; no router port forwarding by default.
- Acceptance: backup/restore and conflict handling are explained before bidirectional sync is enabled.
- Verification: manual backup and recovery walkthrough on local copies.

Dependency Order

Foundation → API and FTS → concrete importer (after archive) and UI/LAN → semantic search and MCP → sync/hardening. UI prototyping can start once the API and minimal model are stable. Qwen, gpt-oss-20b, and Gemma 4 are target local models; OpenCode and Hermes are clients/agents to integrate later.

Out of Scope for the First Slice

- Arbitrary interpretation of the ChatGPT archive before seeing it.
- Exposing the server directly to the internet.
- Autonomous agent writes of “memories” without approval and provenance.
- Choosing an Android replication format before measuring performance and the real PWA experience.

Personal Memory Checklist

The detailed plan is in plan.md. The initial product comparison is in existing-solutions-comparison.md. The user approved building our own core where existing solutions do not meet the requirements.

Preparation

- [x] Confirm the repository and existing specification.
- [x] Compare documented existing products and record requirement gaps.
- [x] User confirmed custom development if no complete solution exists; reuse UI/adapters where useful.

Phase 0 - Evaluate Existing Products

- [x] Compare Khoj, Open WebUI, AnythingLLM, Memos, Mem0, and Letta against priority requirements.
- [ ] Manually prototype with synthetic data for the most suitable candidates.
- [x] Recommend an interoperable custom core; evaluate existing products as UIs/clients/adapters.
- [x] User approved the custom direction on 2026-10-01.

Phase 1 - Foundation and Contracts

- [ ] Inspect the available runtime and repository instructions.
- [ ] Define RAW / INDEX / DERIVED contracts and RAW preservation rules.
- [ ] Define the initial schema and versioned migrations, including attachment binary metadata, hashes, and source links.
- [ ] Document local initialization and backup.
- [ ] Manual acceptance review.

Phase 2 - API and Lexical Search

- [ ] Versioned local API.
- [ ] FTS5 index and provenance in results.
- [ ] Documented generic import without assumptions about the ChatGPT export.
- [ ] Manual walkthrough with synthetic data.

Phase 3 - ChatGPT Import (when the archive is available)

- [ ] Inspect the archive and inventory its real formats.
- [ ] Implement an adapter based on the observed format.
- [ ] Preserve RAW and ensure retryable import with a results report, including attached images/files and their source-message links.
- [ ] Verify locally and privately using a copy of the archive; do not put data or results in Git.

Phase 4 - UI and Android on LAN

- [ ] Search, exploration, and provenance UI.
- [ ] Entity and relationship map.
- [ ] Image/file previews, original access, and offline access to synced attachments on Android.
- [ ] Local Android access and a message for when the PC is unavailable.
- [ ] Manual PC + Android walkthrough.

Phase 5 - Semantic Search

- [ ] Replaceable embeddings provider and version metadata.
- [ ] Integrate local models Qwen, gpt-oss-20b, and Gemma 4 through compatible runtimes.
- [ ] Combined FTS + semantic search.
- [ ] Manual walkthrough or documentation of missing local dependencies.

Phase 6 - MCP and Adapters

- [ ] Initially read-only MCP bridge.
- [ ] Adapters for ChatGPT and the OpenCode and Hermes agent harnesses.
- [ ] Validation and logging before any write operation.
- [ ] Manual connection in each supported client.

Phase 7 - Synchronization and Recovery

- [ ] Design PC-authoritative replication and conflict handling.
- [ ] Document LAN access and backup/restore.
- [ ] Manual restoration walkthrough.